Assignment 3

Test Report

NAME: Lamisa Aziz
NSID: zlq765
STUDENT ID: 11380653
CLASS: CMPT_353

1 Backend Testing (Node.js + Express)

I tested the backend using a combination of:

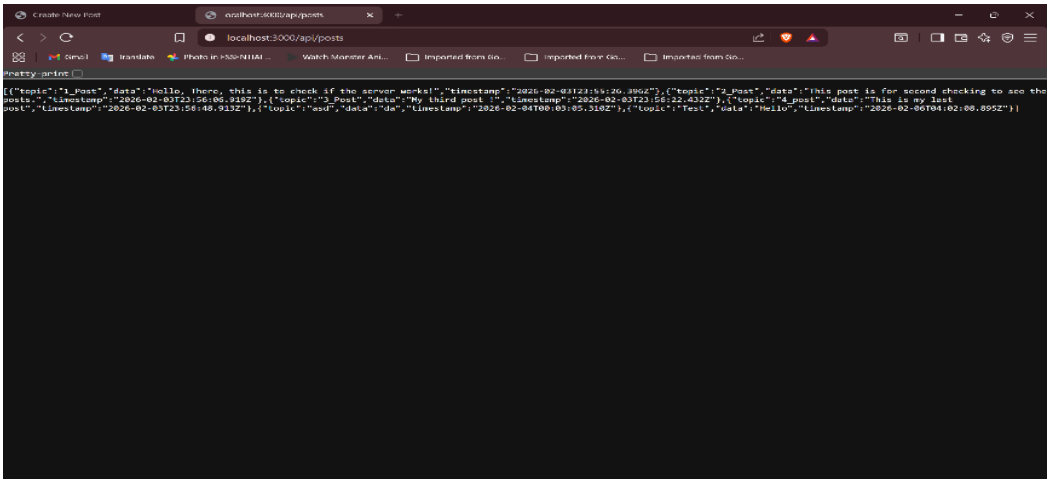
• Direct browser requests

- http://localhost:3000/api/posts Verified that the server returns valid JSON and loads existing posts from posts.json.

• Manual POST requests using curl

- Example:

```
curl.exe -X POST http://localhost:3000/api/posts `
>>   -H "Content-Type: application/json" `
>>   -d '{\"topic\":\"Test\",\"data\":\"Hello\"}'
```

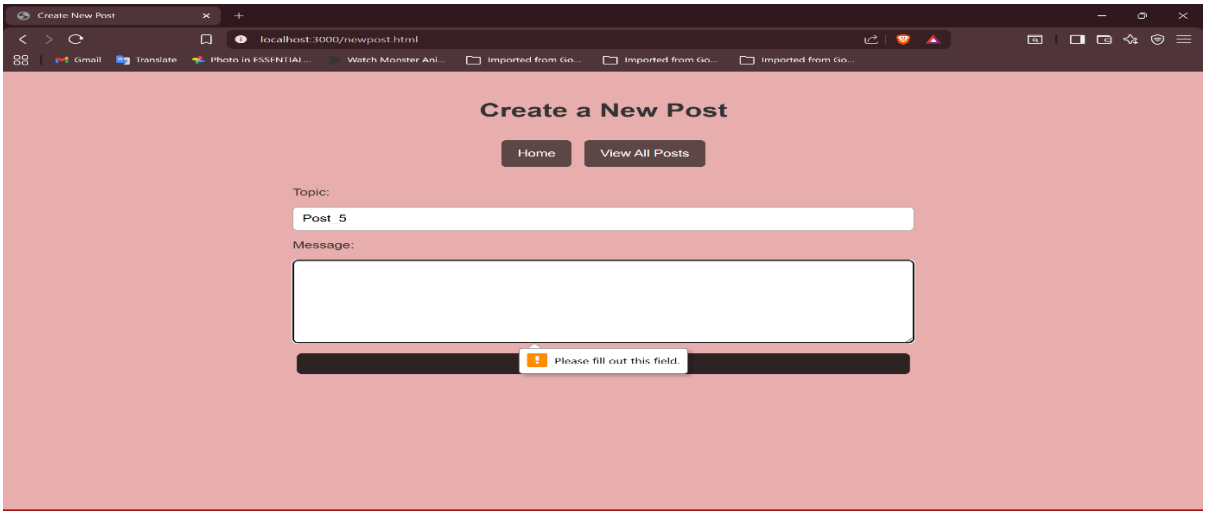


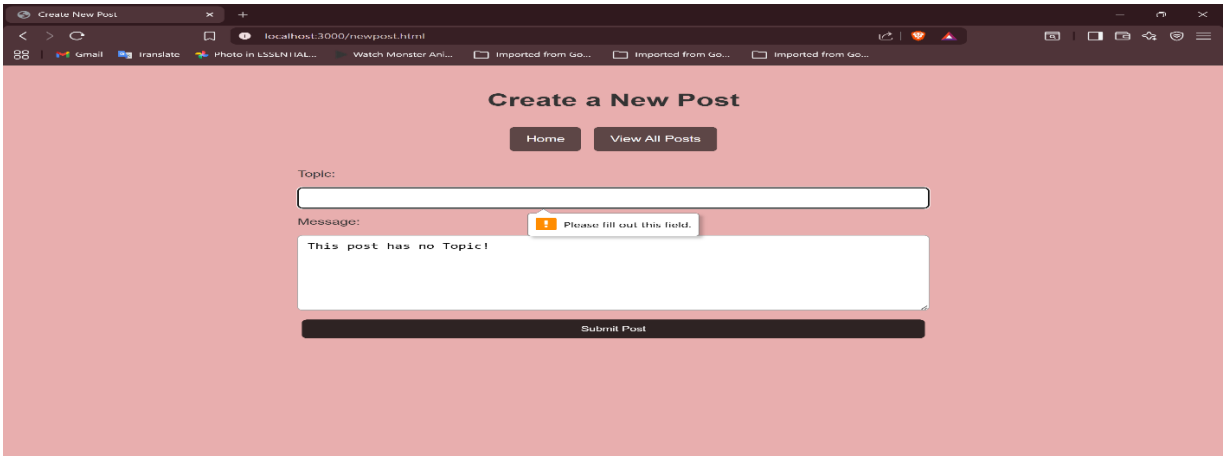
Correct status codes
Proper JSON structure
Timestamp generation
Persistence to data/posts.json

- **Error handling tests**

I intentionally sent invalid requests:

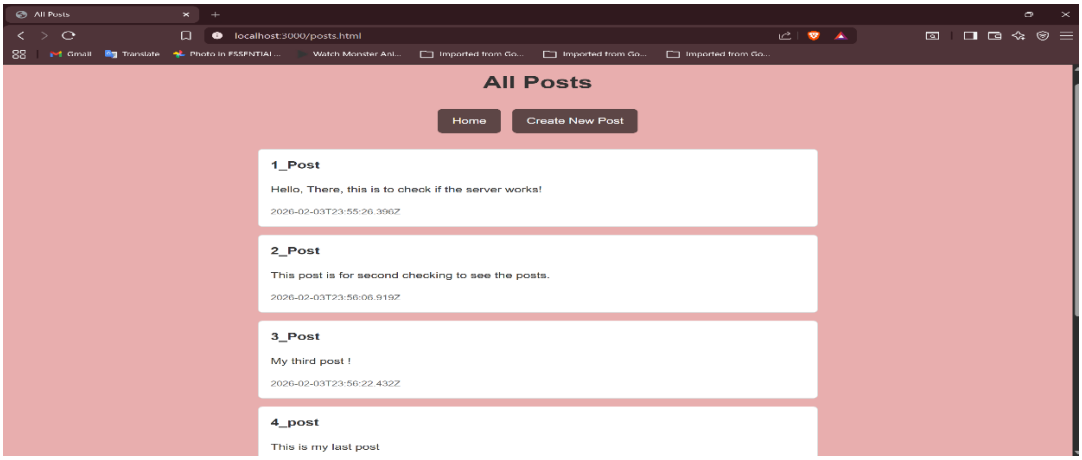
- Missing topic
- Missing data
- Empty JSON body

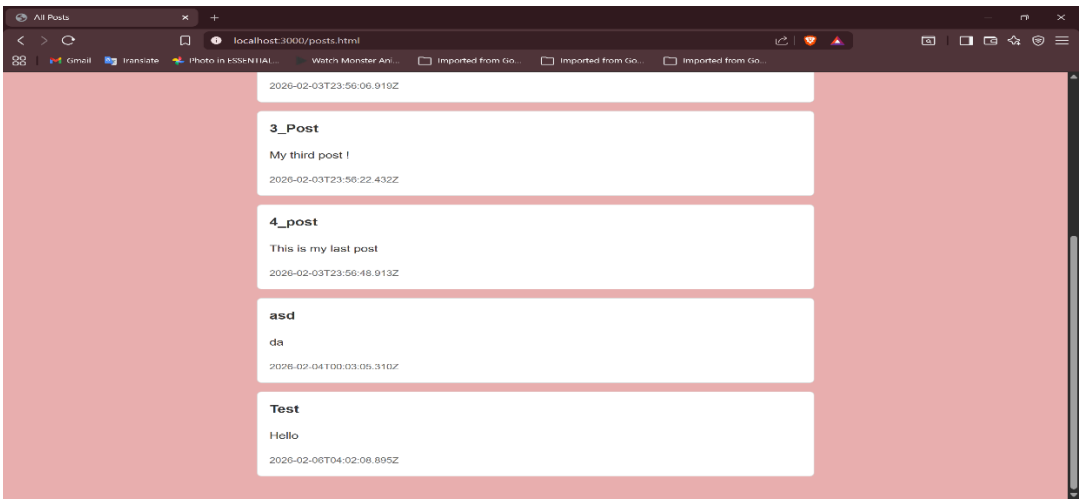




- **Persistence verification**

- Restarted the server
- Confirmed posts remained in posts.json
- Confirmed GET still returned the same posts





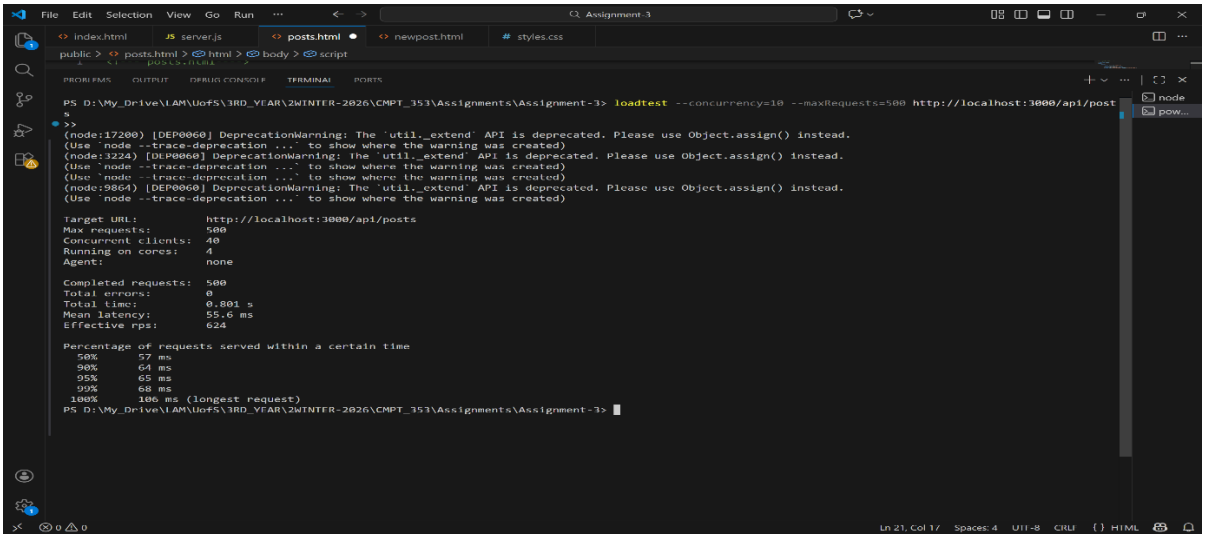
2 Frontend Testing (HTML + JS)

- **newpost.html**
  - Submitted valid posts -> “Post created!” message appeared
  - Form cleared automatically
  - Verified the new post appeared in posts.json
  - Verified the new post appeared in posts.html
- **posts.html**
  - Confirmed posts load immediately
  - Confirmed auto-refresh every 3 seconds using setInterval
  - Verified new posts appear without reloading the page
  - Verified empty state message (“No posts yet.”)
- **index.html**
  - Verified navigation buttons work
  - Verified CSS styling loads correctly

3. Load Testing Results

I used the loadtest tool to evaluate the performance of my backend under concurrent load. The command I used was:

- loadtest --concurrency=10 --maxRequests=500 http://localhost:3000/api/posts



Analysis

The server performed reliably under load. Even with 40 concurrent clients and 500 total requests, the backend returned **zero errors** and maintained low latency. The high effective throughput (624 requests/second) indicates that the GET endpoint is efficient and that file-based reads do not significantly degrade performance under moderate load. Overall, the system remained stable and responsive throughout the test.

4.Challenges and Solutions

Challenge 1 - Express module not found

Issue: Running node server.js produced “Cannot find module ‘express’”.

Solution: Initialized project with npm init -y and installed Express using npm install express.

Challenge 2 - Wrong port when testing

Issue: Tried accessing localhost:80 while running locally.

Solution: Realized port 80 is only for Docker. Local testing uses localhost:3000.